Chapter 11

Exercise 2

Statement. We conduct a study with $n = 4$ participants who have just purchased cell phones, in order to model the time until phone replacement. The first participant replaces her phone after 1.2 years. The second participant still has not replaced her phone at the end of the two-year study period. The third participant changes the phone number and is lost to follow up (but has not yet replaced her phone) 1.5 years into the study. The fourth participant replaces her phone after 0.2 years. For each of the four participants ($i = 1, \ldots, 4$), answer the following questions using the notation introduced in Section 11.1.1:

(a) Is the participant's cell phone replacement time censored?

(b) Is the value of c_i known, and if so, then what is it?

(c) Is the value of t_i known, and if so, then what is it?

(d) Is the value of y_i known, and if so, then what is it?

(e) Is the value of δ_i known, and if so, then what is it?

Solution

We use standard right-censoring notation:

$$y_i = \min(t_i, c_i), \qquad \delta_i = \mathbf{1}\{t_i \leq c_i\},$$

where t_i is the true event time (replacement time) and c_i is the censoring time.

Participant 1: Replaces after 1.2 years.

- (a) Not censored (event observed).
- (b) c_1 is not observed exactly; we only know $c_1 > 1.2$.
- (c) t_1 is known: $t_1 = 1.2$.
- (d) y_1 is known: $y_1 = \min(t_1, c_1) = 1.2$.
- (e) δ_1 is known: $\delta_1 = 1$.

Participant 2: No replacement by study end (2 years).

- (a) Censored (right-censored at study end).
- (b) c_2 is known: $c_2 = 2$.
- (c) t_2 is not known; we only know $t_2 > 2$.

- (d) y_2 is known: $y_2 = \min(t_2, c_2) = 2$.
- (e) δ_2 is known: $\delta_2 = 0$.

Participant 3: Lost to follow-up at 1.5 years without replacement.

- (a) Censored (right-censored at loss to follow-up).
- (b) c_3 is known: $c_3 = 1.5$.
- (c) t_3 is not known; we only know $t_3 > 1.5$.
- (d) y_3 is known: $y_3 = \min(t_3, c_3) = 1.5$.
- (e) δ_3 is known: $\delta_3 = 0$.

Participant 4: Replaces after 0.2 years.

- (a) Not censored (event observed).
- (b) c_4 is not observed exactly; we only know $c_4 > 0.2$.
- (c) t_4 is known: $t_4 = 0.2$.
- (d) y_4 is known: $y_4 = \min(t_4, c_4) = 0.2$.
- (e) δ_4 is known: $\delta_4 = 1$.

Exercise 10 (Brain Tumor Data)

Statement. This exercise focuses on the brain tumor data, which is included in the ISLP library.

(a) Plot the Kaplan–Meier survival curve with ± 1 standard error bands, using the `KaplanMeierFitter()` estimator in the `lifelines` package.

(b) Draw a bootstrap sample of size $n = 88$ from the pairs (y_i, δ_i), and compute the resulting Kaplan–Meier survival curve. Repeat this process $B = 200$ times. Use the results to obtain an estimate of the standard error of the Kaplan–Meier survival curve at each timepoint. Compare this to the standard errors obtained in (a).

(c) Fit a Cox proportional hazards model that uses all of the predictors to predict survival. Summarize the main findings.

(d) Stratify the data by the value of `ki`. (Since only one observation has `ki==40`, you can group that observation with the observations that have `ki==60`.) Plot Kaplan–Meier survival curves for each group. Describe the differences.

Solution (code + interpretation guide)

Setup and data

The tasks are most naturally completed in Python using `lifelines`. The code below assumes the dataset has:

- a duration column (the observed time) called `time` (this corresponds to y_i),
- an event indicator column called `status` (this corresponds to δ_i, with $1 =$ event, $0 =$ censored),
- additional predictors including `ki`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from lifelines import KaplanMeierFitter, CoxPHFitter

from ISLP import load_data
df = load_data("BrainCancer")

df = df.dropna().reset_index(drop=True)

time_col = "time"     # y_i
event_col = "status"  # delta_i (1=event, 0=censored)
```

(a) Kaplan–Meier curve with ± 1 standard error bands

Greenwood's formula yields an estimated variance for $\widehat{S}(t)$ at the event times. We plot $\widehat{S}(t) \pm 1 \cdot \mathrm{SE}(\widehat{S}(t))$, clipping to $[0, 1]$.

```
kmf = KaplanMeierFitter()
kmf.fit(durations=df[time_col], event_observed=df[event_col], label="KM
    estimate")

S = kmf.survival_function_["KM estimate"
var = kmf.variance_["KM estimate"]
se = np.sqrt(var)

upper = np.minimum(1.0, S + se)
lower = np.maximum(0.0, S - se)

plt.figure()
plt.step(S.index, S.values, where="post")
plt.step(upper.index, upper.values, where="post", linestyle="--")
plt.step(lower.index, lower.values, where="post", linestyle="--")
plt.ylim(0, 1.02)
plt.xlabel("Time")
plt.ylabel("Survival probability")
plt.title("Kaplan Meier with 1 SE bands (Greenwood)")
plt.show()
```

What to report. The KM curve is a non-increasing step function. The bands typically widen over time because the "risk set" shrinks and uncertainty increases.

(b) Bootstrap standard errors for $\widehat{S}(t)$

We bootstrap the *pairs* (y_i, δ_i) by sampling rows with replacement, fit KM each time, and then compute the empirical standard deviation of $\widehat{S}(t)$ at each timepoint. To compare curves meaningfully, we evaluate them on a common time grid.

```
B = 200
n = df.shape[0]

grid = np.sort(df[time_col].unique())
```

```
boot_S = np.zeros((B, len(grid)))

for b in range(B):
    idx = np.random.randint(0, n, size=n)
    dboot = df.iloc[idx].reset_index(drop=True)

    kmf_b = KaplanMeierFitter()
    kmf_b.fit(dboot[time_col], event_observed=dboot[event_col])

    boot_S[b, :] = kmf_b.predict(grid).values

boot_se = boot_S.std(axis=0, ddof=1)

greenwood_se_on_grid = np.interp(grid, se.index.values, se.values, left=se
    .values[0], right=se.values[-1])

plt.figure()
plt.plot(grid, boot_se, label="Bootstrap SE")
plt.plot(grid, greenwood_se_on_grid, linestyle="--", label="Greenwood SE")
plt.xlabel("Time")
plt.ylabel("Standard error of KM estimate")
plt.title("Bootstrap vs Greenwood SE")
plt.legend()
plt.show()
```

Comparison. The bootstrap SE and Greenwood SE are often similar in shape. Differences can appear at later times (few individuals remain at risk) and because the bootstrap captures sampling variability in a slightly different way than Greenwood's large-sample approximation.

(c) Cox proportional hazards model using all predictors

Fit a Cox model with all predictors. You typically:

- one-hot encode categorical variables,
- interpret coefficients via hazard ratios $\exp(\beta)$,
- check which predictors are significant and their direction.

```
X = df.drop(columns=[time_col, event_col])

X = pd.get_dummies(X, drop_first=True)

df_cox = pd.concat([df[[time_col, event_col]], X], axis=1)

cph = CoxPHFitter()
cph.fit(df_cox, duration_col=time_col, event_col=event_col)

cph.print_summary()
```

What to write in the summary.

- Identify predictors with small p-values (strong evidence of association with survival).

- Report hazard ratios (HR): HR > 1 indicates higher hazard (shorter survival), HR < 1 indicates lower hazard (longer survival).
- Note limitations: correlated predictors can inflate standard errors; proportional hazards is an assumption.

(d) Stratify by `ki` and plot KM curves

Create strata by grouping `ki==40` with `ki==60`, then plot separate KM curves.

```
df_strat = df.copy()
df_strat["ki_stratum"] = df_strat["ki"].replace({40: 60})

plt.figure()
for g, dfg in df_strat.groupby("ki_stratum"):
    km = KaplanMeierFitter()
    km.fit(durations=dfg[time_col], event_observed=dfg[event_col], label=f
        "ki={g}")
    km.plot_survival_function(ci_show=False)

plt.ylim(0, 1.02)
plt.xlabel("Time")
plt.ylabel("Survival probability")
plt.title("Kaplan Meier curves by ki stratum (40 grouped with 60)")
plt.show()
```